Architectural Design of StockComparer

Sprint 3

Matteo Organek **ORG23613773**

Oleksi Ivanov **A00034498**

Obaida Muqbel **A00013570**

Nilay Oral **Z23607537**

1. Introduction

2. System Overview

3. Compound Components

3.1 GraphSystem

3.2 DataProviderSystem

3.3 ProfileManagementSystem

3.4 WatchlistSystem

3.5 PersistenceSubsystem

4. Domain-Independent Architectural Styles

4.1 Model–View–Controller Architecture

4.2 Layered Architecture

4.3 Adapter Pattern

4.4 Strategy Pattern

5. Service-Oriented Architecture Alignment

6. Persistence and Offline Support

7. External APIs and Libraries

8. Indicator Framework Architecture

9. Team Coordination and Meetings

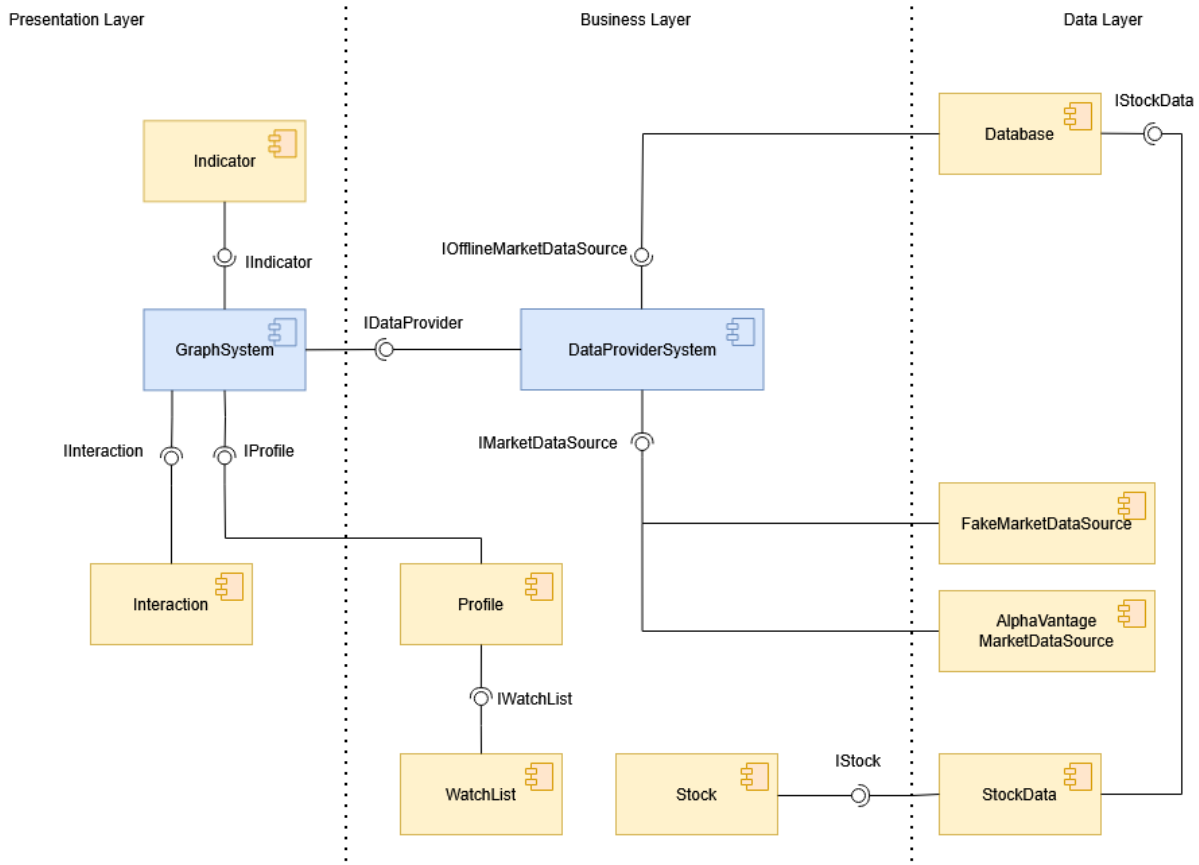
10. Conclusion

1. Introduction

This report presents the architectural refinement of the **StockComparer system** carried out during Sprint 3 of the development lifecycle. The purpose of this sprint was to define compound components, strengthen subsystem boundaries, and demonstrate the use of domain-independent architectural styles supporting modularity, extensibility, and offline functionality.

The system is designed as a desktop-based stock comparison and visualization platform that integrates external market data providers, supports offline persistence through local storage, and allows users to manage profiles and watchlists while interacting with graphical financial visualizations.

Sprint 3 extends the structural decisions introduced during Sprint 2 by formalising subsystem composition and introducing interface-based service abstractions across the architecture. The high-level architecture of the system is illustrated in **Figure 1**, which presents the layered organisation of the application.



2. System Overview

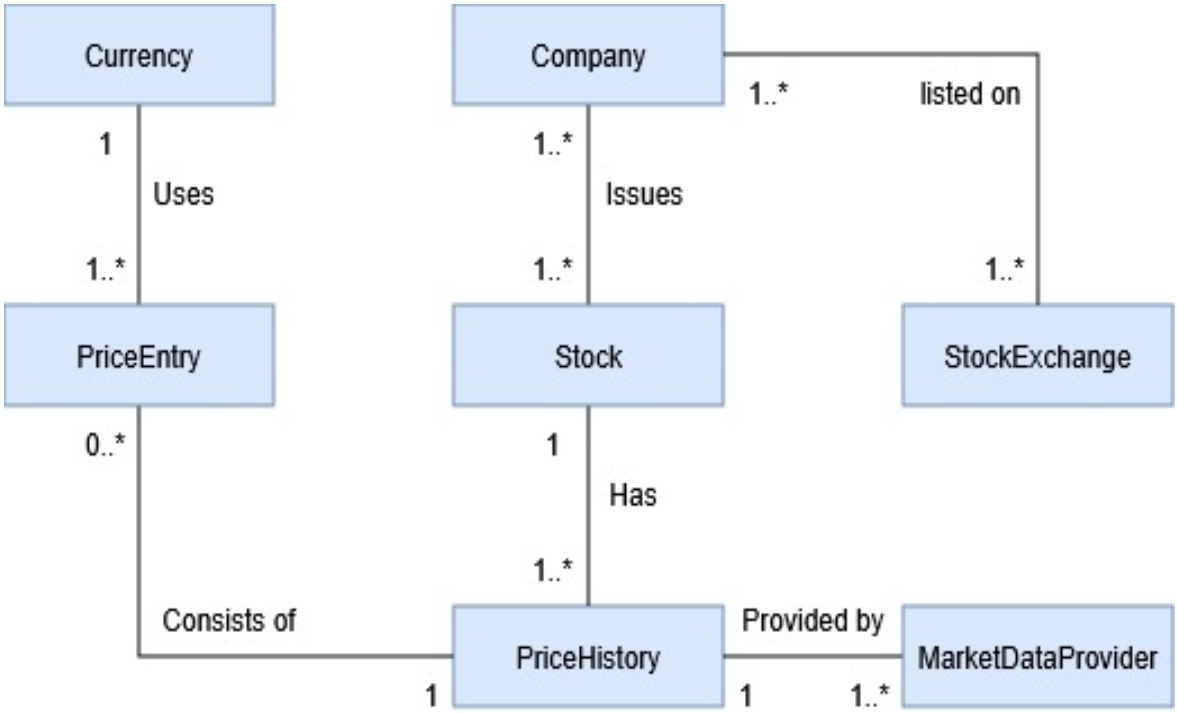
The StockComparer architecture is organised into cooperating compound components responsible for distinct responsibilities within the application.

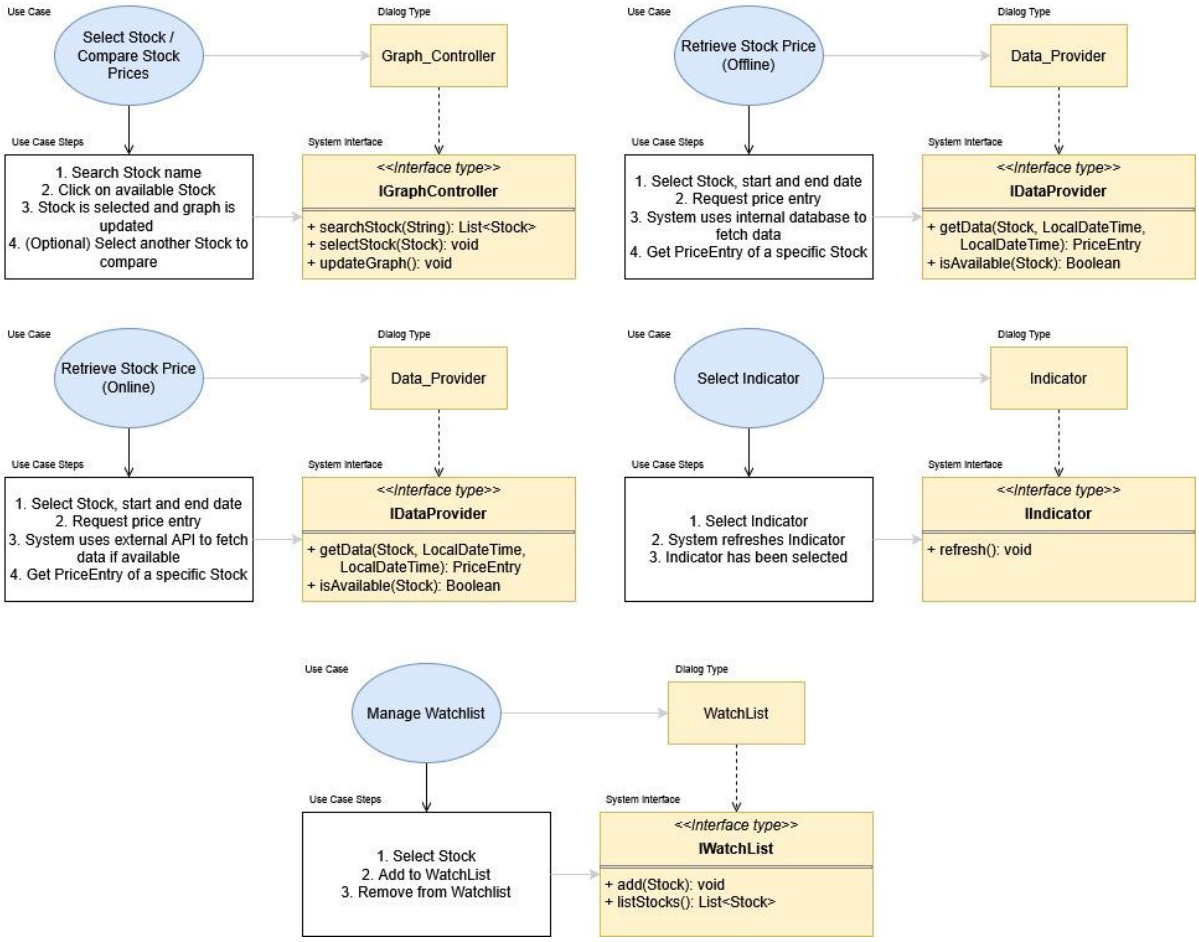
These components include:

- GraphSystem
- DataProviderSystem
- ProfileManagementSystem
- WatchlistSystem
- PersistenceSubsystem
- ApplicationControllerSystem

Each subsystem encapsulates related classes and interfaces and communicates with other components through clearly defined service contracts.

The relationship between business entities such as stocks, companies, and user profiles is illustrated in the **Business Concept Model (Figure 2)**, which provides the domain-level abstraction supporting the application structure.


Currency
Company
1..*
listed on
1
Uses
1..*
1..*
Issues
1..*
1..*
PriceEntry
Stock
StockExchange
0..*
1
Has
1..*
Consists of
PriceHistory
Provided by
MarketDataProvider
1
1
1..*



The interaction between subsystems and external services is further illustrated in the **System Interfaces Diagram (Figure 3)**.

3. Compound Components

3.1 GraphSystem Component

The GraphSystem is responsible for visualising stock market data and supporting user interaction with graphical chart representations.

This subsystem includes:

- Graph
- IGraph
- Indicator
- Interaction
- GraphController
- graph-view.fxml

The GraphSystem operates as a compound visualization component where controllers coordinate data retrieval and rendering through interface-based abstraction.

The IGraph interface separates controller logic from rendering implementation, allowing independent extension of visualization functionality without affecting the rest of the application.

The structure of this subsystem corresponds to the component-level decomposition shown in **Figure 4**, where graph-related services are grouped as a cohesive visualization module.

Additionally, the GraphSystem integrates an extensible indicator architecture through the IIndicator interface. Although technical indicators such as moving averages or RSI overlays are not yet implemented, the architecture allows these features to be integrated without modification of the rendering pipeline.

This demonstrates forward-compatible extensibility.

3.2 DataProviderSystem Component

The DataProviderSystem is responsible for retrieving stock market data from both external and internal sources.

It includes:

- IMarketDataSource
- YahooFinanceAPI
- AlphaVantageAPI
- FakeMarketDataSource
- ApiBridge
- StockPriceDataSource
- StockExchange

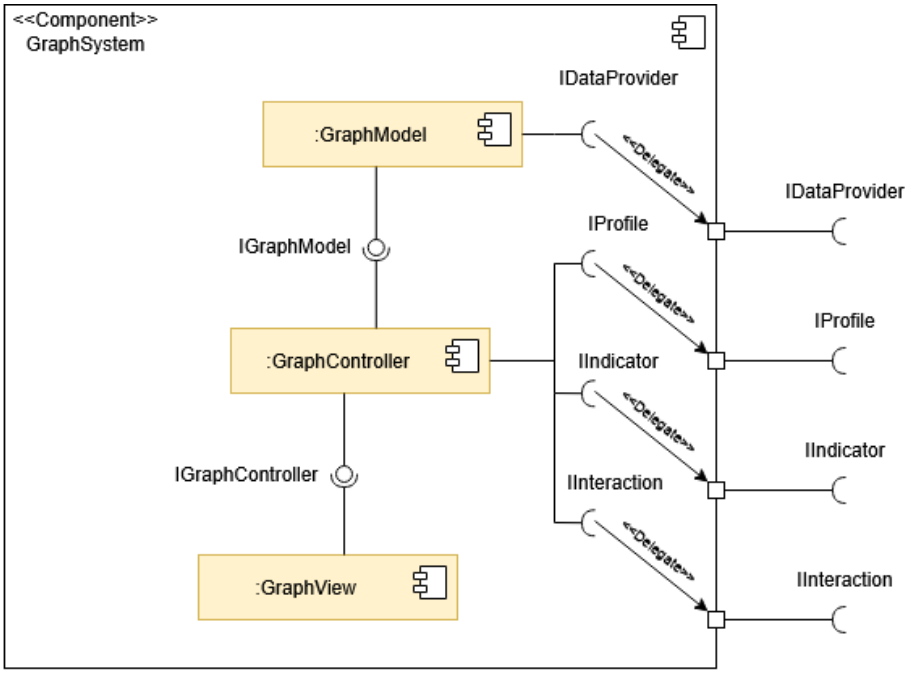
- The use of the IMarketDataSource interface allows runtime selection between different data providers, enabling the system to switch between online and offline data retrieval modes dynamically.

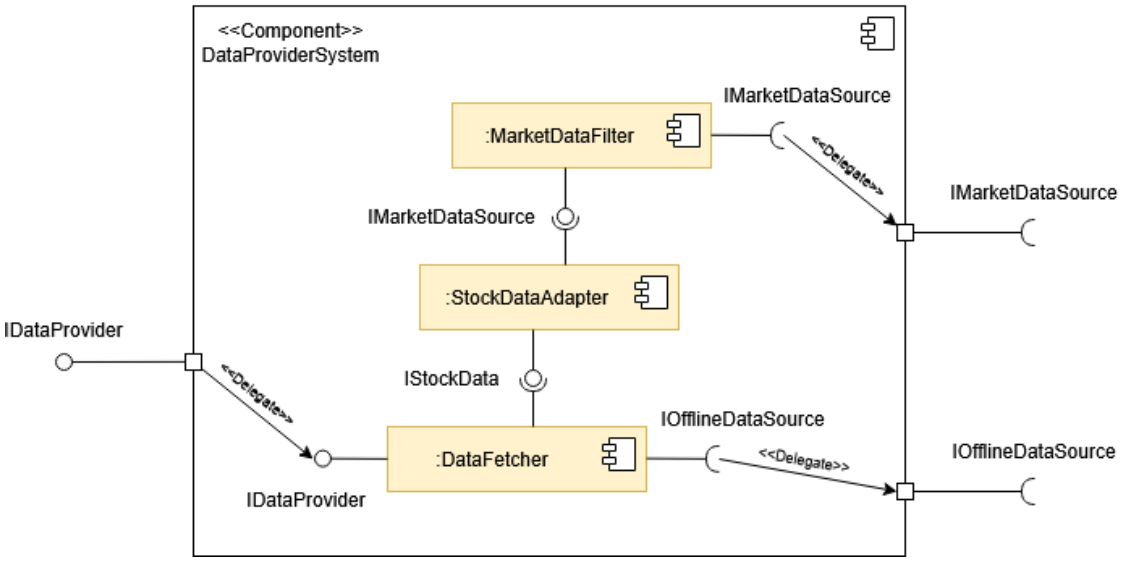
The ApiBridge component converts external API responses into internal data model structures used by the application.

This ensures:

- independence from third-party APIs
- improved portability
- testability through mock providers
- support for offline operation

The interaction between the DataProviderSystem and external services is illustrated in **Figure 3**, where API integration boundaries are clearly defined.


<<Component>>
GraphSystem
IDataProvider
:GraphModel
<<Delegate>>
IDataProvider
IProfile
IGraphModel
<<Delegate>>
IProfile
:GraphController
IIndicator
<<Delegate>>
IIndicator
IGraphController
IInteraction
<<Delegate>>
:GraphView
IInteraction


<<Component>>
DataProviderSystem
IMarketDataSource
:MarketDataFilter
<<Delegate>>
IMarketDataSource
IMarketDataSource
:StockDataAdapter
IDataProvider
<<Delegate>>
IStockData
IOfflineDataSource
:DataFetcher
<<Delegate>>
IOfflineDataSource
IDataProvider

3.3 ProfileManagementSystem Component

The ProfileManagementSystem manages user-specific configuration data and supports persistent storage of preferences.

This subsystem includes:

- Profile
- IProfile
- JSON-based persistence storage
- profile configuration services

Profiles are stored locally using JSON serialization inside the project data directory, enabling offline accessibility and session restoration functionality.

The persistence workflow supporting profile management is aligned with the data abstraction structure illustrated in **Figure 2**, where user-level entities are represented alongside stock-domain objects.

3.4 WatchlistSystem Component

The WatchlistSystem enables users to track selected stocks across sessions and maintain personalised monitoring lists.

This subsystem includes:

- Watchlist
- IWatchlist
- profile-linked persistence storage

Watchlists are associated with user profiles and stored locally using JSON serialization.

This ensures:

- offline accessibility
- persistence across sessions
- personalised monitoring capability

Integration between watchlists and profiles follows the relationships defined in **Figure 2**, reinforcing the domain model consistency across persistence and visualization layers.

3.5 PersistenceSubsystem Component

Persistence within the system is implemented using structured JSON storage combined with internal database abstraction classes.

Key persistence classes include:

- Database
- IDataProvider
- PriceHistory
- PriceEntry
- Stock
- Company
- Currency

Persistence supports:

- profile storage
- watchlist storage
- cached historical stock data
- offline fallback operation

This subsystem ensures continued usability of the application even when external data providers are unavailable.

Its role within the layered architecture is illustrated in **Figure 1**, where persistence services operate beneath controller-level coordination logic.

---

4. Domain-Independent Architectural Styles

4.1 Model–View–Controller Architecture

The StockComparer system follows the Model–View–Controller architectural pattern using JavaFX.

Model layer:

- Stock
- Profile
- Watchlist
- PriceHistory
- Indicator

View layer:

- graph-view.fxml
- main-view.fxml
- CSS styling resources

Controller layer:

- GraphController
- ChartMetadataController
- Application controllers

MVC architecture separates presentation logic from business logic and supports independent development of user interface components.

The separation of MVC layers is reflected in the system structure illustrated in **Figure 1**.

4.2 Layered Architecture

The application follows a layered architecture consisting of:

Application Layer
Controller Layer
Model Layer
Data Access Layer
External Service Layer

Each layer communicates through interface abstractions rather than direct implementation coupling.

Benefits include:

- improved modularity
- simplified debugging
- enhanced maintainability
- improved testing capability

Layer relationships are illustrated in **Figure 1**, where controller services mediate communication between presentation and persistence layers.

4.3 Adapter Pattern

External financial data providers are integrated using the Adapter pattern.

Examples include:

- YahooFinanceAPI
- AlphaVantageAPI
- ApiBridge

The adapter converts external data structures into internal domain models used by the system.

This ensures architectural independence from third-party libraries and supports replacement of providers without modification of controller logic.

Adapter boundaries are illustrated in **Figure 3**

4.4 Strategy Pattern

The system dynamically selects between alternative data providers using the Strategy pattern.

Examples include:

- YahooFinanceAPI (online provider)
- AlphaVantageAPI (online provider)
- FakeMarketDataSource (test offline provider)

Provider switching is handled through the IMarketDataSource abstraction.

This enables flexible runtime configuration without modifying higher-level application logic.

5. Service-Oriented Architecture Alignment

Although the system is implemented as a desktop application, it follows service-oriented architectural principles through interface-based service abstraction.

Examples include:

- IMarketDataSource
- IDataProvider
- IGraph
- IIndicator
- IProfile
- IWatchlist

These interfaces define service contracts between subsystems and enable independent replacement of implementations.

This improves flexibility and supports future migration toward distributed architectures if required.

Subsystem service relationships are illustrated in **Figure 4**, where compound components communicate through interface boundaries.

---

6. Persistence and Offline Support

Offline functionality is supported through JSON-based persistence storage combined with fallback data providers.

Profiles and watchlists are stored locally inside:

data/profiles/

Offline capability is achieved through:

- FakeMarketDataSource
- JSON persistence storage
- cached historical stock structures

This ensures that users can continue interacting with the system without internet connectivity.

Offline support integration is illustrated in **Figure 3**, where fallback provider interaction is represented alongside API services.

7. External APIs and Libraries

The system integrates several external libraries supporting modular architecture and rapid development.

Examples include:

JavaFX – graphical interface rendering
YahooFinanceAPI – live stock data retrieval

AlphaVantageAPI – live stock data retrieval
Jackson / Gson – JSON serialization
ControlsFX – enhanced UI components

External libraries are integrated through adapter layers to preserve independence between third-party dependencies and internal domain structures.

8. Indicator Framework Architecture

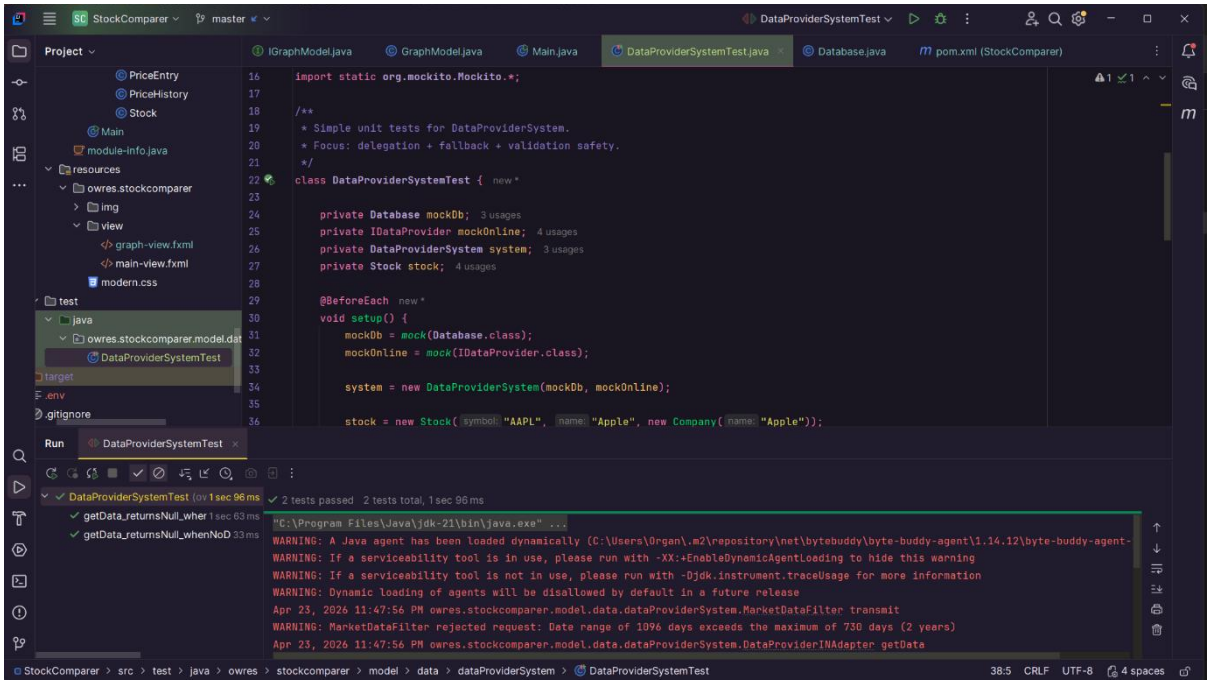
The system includes an extensible indicator framework implemented through the IIndicator interface.

Although concrete technical indicators such as moving averages or RSI overlays are not yet implemented, the architecture supports integration of analytical overlays without modification of the graph rendering subsystem.

This demonstrates forward-compatible extensibility consistent with compound component design principles illustrated in **Figure 4**.

10. Test cases

Test cases have been implemented to check different things such as non null assumptions mainly in DataProviderSystem:



10. Team Coordination and Meetings

Date	Topic	Outcome
08/02	Kick-off	Project scope defined
16/02	Architecture	Components and controllers agreed
22/02	Planning	Missing subsystems identified
23/02	Design	Diagrams refined and responsibilities assigned
24/02	Review	Progress evaluated and adjustments applied
04/03	Implementation	Data layer issues identified
15/03	Architecture	Transition toward Clean Architecture
15/04	Final review	Final structure validated.

These meetings ensured architectural consistency across all development phases.

11. Conclusion

Sprint 3 successfully formalised the compound component structure of the StockComparer system while reinforcing layered architecture principles and service-oriented abstraction across subsystems.

The system demonstrates strong modularity through interface-based service contracts, flexible provider substitution through strategy-based data sources, and offline persistence through JSON storage mechanisms.

The architecture supports future extensibility through indicator frameworks and replaceable service providers, ensuring maintainability and scalability for subsequent development iterations.